

Analysis of Minimax Adversarial Search Optimizations in Connect 4

Jacob A. Geldbach
College of Engineering: CS
University of Idaho
Moscow Idaho, United States
geld8788@vandals.uidaho.edu

Abstract—Adversarial search is a topic of search related to zero-sum scenarios between two opponents with directly opposing goals. The goal of this project was to implement and analyze the Minimax algorithm in the context of a game of Connect 4. The Minimax implementation is used for the AI player to generate the best possible move it can make given the current board state. In the scenario of an AI player vs an AI player both players utilized the exact same Minimax implementation to find the next best move. This implementation allows for Connect 4 to be played with an AI opponent of varying difficulty where difficulty is set by the number of turns the AI player is allowed to look ahead in the Connect 4 Minimax built game tree. The project includes multiple different adversarial search algorithm optimizations that are able to be included or excluded from a specific run of the program, allowing for comparison of the collected analytical data. Alpha-beta Pruning, Iterative Deepening combined with Move Re-ordering and Selective Deepening (Quiescent Search) were all used on top of the base Minimax algorithm to further improve its efficiency in regards to time per turn and nodes explored.

Index Terms—Adversarial, Search, Minimax, Connect4, Efficiency, Alpha, Beta, Pruning, Quiescent

I. ALGORITHM

Minimax is an algorithm based on mutually recursive calls to a minMove and a maxMove function. Each layer of the recursive calls is effectively the AI looking one more turn ahead. And due to the adversarial environment that this algorithm is built for, each layer is finding either the best move for that player or the worse move for that player given the current board state. The board state is also evolving as the algorithm works down to the max depth for the call, as each layer is calling the min/maxMove() function for each of the possible valid moves at that layer. When the recursive calls hit the max depth that Minimax was called with, an evaluation of the board with that path of prospective moves is returned up the tree. This evaluation is a heuristic, as if the algorithm has reached this far in the search an estimation of how good or bad this board state is for the player must be returned. The evaluation function I used in this project is defined below in more detail. Each layer of the recursion all final states of the board are checked such as a win for the player, a loss for the player or the board has filled up entirely and it is to end in a draw. When these states are found the extremes of the board value spectrum (-inf, inf) are returned to influence the Minimax towards or away from them (win or lose). Once each mutually recursive call of min/maxMove pertaining to one of

the current valid possible moves returns in the root Minimax function, each value is compared and the move corresponding with the highest value is returned and officially played by the AI for that turn. The time efficiency of this Minimax implementation is $O(b^d)$ nodes processed per turn of the AI, where b is number of valid moves or columns and d is depth of tree look ahead or turns the AI can look ahead and. The space efficiency is much better at $O(b*d)$ since only the exact path down to the current node or the current recursion stack of nodes is kept in memory at a single time.

A. Evaluation Function

The evaluation function used in this project was a tally of the entire boards "score". Where each individual window, defined as 4 spaces in a row vertically, horizontally and diagonally, is scored based on how "good" or likely a player is to win or lose with that number of their pieces in a single window. There are a total of 69 total scorable windows, leading this evaluation function to be space and time $O(1)$ efficient. Most importantly the score for a single window is significantly better for each consecutive piece a player has in a non-contested window. So for example a single piece in a non-contested window (meaning only their pieces and empty slots make up the window) is good, but two of their pieces is scored quadratically more opposed to just 2 times the window with 1 piece. As soon as a window becomes contested it is worth 0 points towards the total evaluation of that board as there is no way to win or lose via that window. Non-contested windows with 3 of a players pieces are valued far more than 2 piece windows because that window is also a win condition on the next turn. All of these scores are the negated values for when it is the opponents pieces instead of the players pieces. With the exception that opponent 3 piece windows are negatively valued even more than the 3 piece windows of the player resulting in the AI player prioritizing the blocking of potential winning moves for their opponent so they do not lose the game.

B. Alpha-Beta Pruning

Alpha-Beta Pruning is an optimization technique that was used in this project's implementation. Alpha-Beta Pruning allows for upper and lower bounds to be passed down to each layer of the mutually recursive calls of min/maxMove. What this allows for is in maxMove the lower bound can be checked

against the prospective value being returned by each call to `minMove`, if the value returned from `minMove` is greater than the current lower bound, that call of `maxMove` knows it will never pick anything less than that value so it updates the lower bound to reflect that. The same is done in `minMove` except the inverse and with the upper bound. As these bounds converge or tighten this informs the different layers that as soon as a call of `minMove` sees the upper bound go lower or equal to the lower bound the program can return from that call and essentially prune all remaining (6 down to 1) child nodes for that branch. The reason for this is that `minMove` knows that the maximizer somewhere along the way higher up in the tree has at worst the lower bound. The alpha and beta values are passed down per call to `minMove` and `maxMove` so that alpha lower bound value is inherited from a parent call to `maxMove`. The same but inverse is true about `maxMove` and its relationship with the beta upper bound, it knows that somewhere up above in the tree a minimizer layer has at worst the beta upper bound value meaning it can prune all of that move nodes child branches once it sees the alpha lower bound value become equal to or greater than the beta upper bound value. It is important to note that Alpha-Beta pruning is only an optimization in regards to number of node processing efficiency, meaning it will not alter the algorithm to where it will now start making different decisions regarding what move to make. Minimax will make the same move choice as it would have without using Alpha-Beta pruning except it potentially can make the same choice in a fraction of total nodes processed. Alpha-Beta pruning on average takes Minimax from a time efficiency $O(b^d)$ to $O(b^{3d/4})$. Wherein combined with some of the techniques expanded on in below sections it can reach time efficiencies of $O(b^{d/2})$.

C. Iterative Deepening and Move Re-Ordering

Move Re-Ordering and Iterative Deepening are two more time efficiency optimization techniques used in this project's implementation. These optimizations are used in conjunction with Alpha-Beta pruning in order to accelerate the rate at which branches are pruned early in the search of the game tree by Minimax. The true optimization that is occurring comes from the order in which the move branches of a node are explored. By exploring the higher valued moves first (moves that lead to a higher valued board evaluation), this leads to the Alpha-Beta pruning ranges being tightened earlier in the search, and with a tighter upper and lower bound range, more branches and entire subtrees of the game tree will be pruned off. Move Re-Ordering in this project is used even in the default run of the algorithm. The algorithm is sorting the list of valid column moves by center, or instead of iterating over the possible moves starting at column 0 to column 6, it looks at the central columns first. The reason for this is that it is well known that center pieces make up winning windows much more often than end column piece positions. This Move Re-Ordering alone led to fewer nodes needing to be processed per AI turn. The real efficiency improvement related to Move Re-Ordering came from its pairing with Iterative Deepening.

What this allowed was for a dynamic best order of which moves to explore at the root of the Minimax search. The way this works is instead of following the same static center sorted order of move exploration for the entire d depth Minimax search, the most "interesting" move ordering is calculated after each iterative call to Minimax. The way it was implemented in this project is Minimax with Alpha-Beta pruning and a depth of 1 is run using the center sorted order of move exploration, once that returns, it doesn't just return the best move at that depth; it returns a ranking list of the best moves. Then another Minimax search is called at depth 2 using the new best move ranked order for the order in which it explores the root's moves. And this is repeated until the max depth for the parent call to Minimax is hit.

D. Selective Deepening (Quiescent Search)

The final optimization that was included in this project is Selective Deepening. This is a adversarial search technique that allows for threat states to be marked, and allow that specific search path through the tree to continue another layer past the current max depth limit. In this implementation of Minimax when a game tree search path gets to the max depth and the evaluation function flags that board state as a possible threat, such as a non-contested window already holding three of one color, meaning a win or loss is available on the very next move. Selective Deepening would then allow the search to instead extend its search one layer deeper to further explore that threat and to more importantly thwart any game ending scenarios just beyond the horizon. Thereform the impending win or loss is resolved as a true terminal value of -inf rather than returned up the tree as a heavier weighted negative value. This threat flag is produced by the same window scan the evaluation function already performs, so identifying the threat adds no extra pass over the board. But to be clear this implementation of Selective Deepening does not continue to go further on until the path is "quiet" or non-threat anymore. It simply extends a single layer of depth to protect the AI player from the Horizon Effect. Unlike the previous optimizations Selective Deepening can change the move the AI ultimately plays: by looking one move beyond the horizon it can secure a win or block a loss that a strict fixed-depth search would have evaluated as merely a strong or weak position.

II. ANALYSIS

TABLE I
CONNECT 4 — ALGORITHM ANALYTICS

Depth	Total States Explored	First Move Explored States	Time/Move	Outcome
Minimax				
4	25519	2800	0.502s	Blue win after 32 turns
5	150015	19607	0.958s	Blue win after 32 turns
6	839984	137256	2.985s	Blue win after 38 turns
Minimax + Alpha-Beta				
4	7286	857	0.430s	Blue win after 32 turns
5	36661	3282	0.513s	Blue win after 32 turns
6	43007	14552	0.625s	Blue win after 38 turns
Minimax + Alpha-Beta + Static Move Re-Ordering				
4	5921	188	0.430s	Blue win after 34 turns
5	25189	1091	0.499s	Red win after 27 turns
6	43007	1690	0.547s	Red win after 33 turns
Minimax + Alpha-Beta + Move Re-Ordering and Iterative Deepening				
4	6721	310	0.425s	Blue win after 36 turns
5	29783	1449	0.476s	Draw after 42 turns
6	59725	6585	0.597s	Red win after 37 turns
Minimax + Alpha-Beta + Move Re-Ordering and Iterative Deepening + Selective Deepening				
4	11233	118	0.466s	Red win after 23 turns
5	42111	1393	0.539s	Draw after 42 turns
6	363481	7325	1.311s	Blue win after 36 turns

III. SAMPLE GAMEPLAY

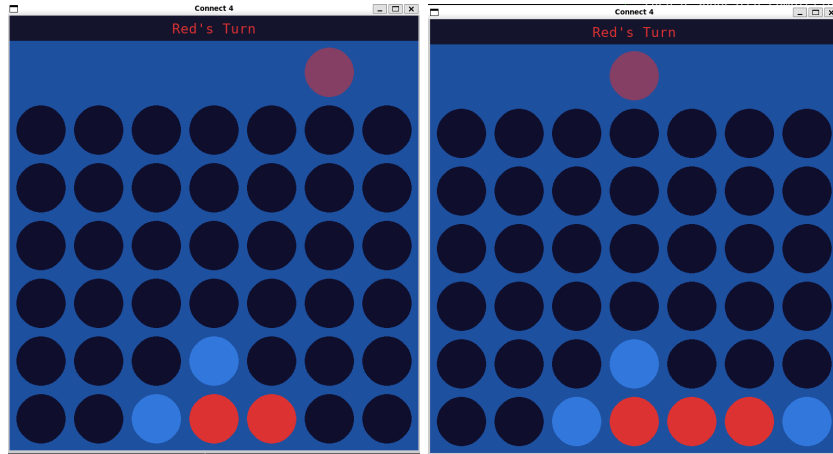


Fig. 1. Before and after the AI (Blue) is able to block the Human (Red) win

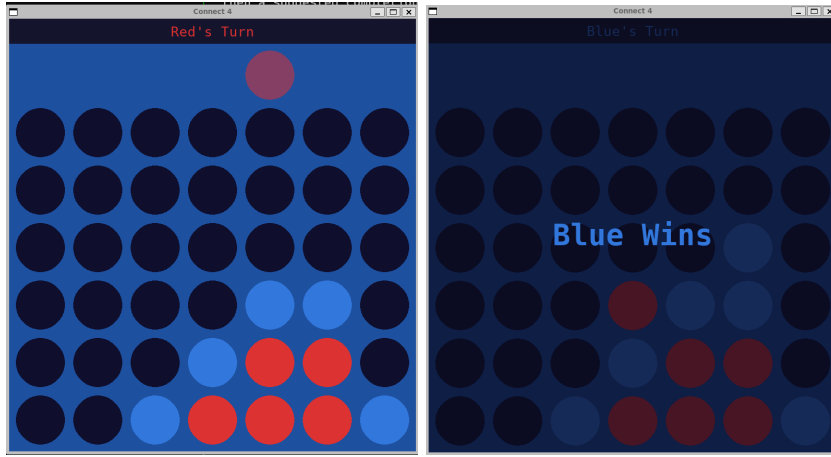


Fig. 2. Before and after the AI (Blue) correct plays the winning move



Fig. 3. Game result AI vs AI depth 5, with and without Selective Deepening

CONCLUSION

This implementation of Minimax showed how well adversarial search can be used to create a strong AI player for an application like Connect 4. The optimization techniques that were implemented also proved to be effective in moves explored as predicted by their theoretical $O(\text{time})$ efficiency calculations. For example, comparing the base Minimax $\text{depth} = 4$ first move search of 2800 moves, which almost exactly the sum of all 4 depths of the tree ($7^0 + 7^1 + 7^2 + 7^3 + 7^4$) to the Minimax $\text{depth} = 4$ with Alpha-Beta pruning first move search of 857 moves, we can see that the number of nodes explored is reduced by a factor of about 3.27, which is consistent with the theoretical improvement from $O(b^d)$ to $O(b^{3d/4})$. Comparing those first two rows of the base Minimax and the Minimax with Alpha-Beta pruning is interesting as all 3 outcomes at the different depths are the same, but the number of nodes or the efficiency of the search is significantly improved with the pruning technique, this is what we expected. We also would have expected the third row to match these three outcomes as well, but due to the fact that the Move Re-Ordering is a static center sorted order as opposed to the first two rows using the left to right static sorting of move exploration per layer, this actually changes which decisions are being made. The way this changes decision making is because the program does nothing other than take the first move in the list currently when it comes to tie breaking. So in the first two rows of the table the left most move is always explored first and therefore chosen in the case of a tie, but in the third row the center most move is explored first and therefore chosen in the case of a tie. Thus explaining that bit of data I at first was not expecting. Currently in this implementation using center static Move Re-Ordering paired with Alpha-Beta pruning is the most efficient of all the tested methods. This is due to the center moves being better Connect 4 moves is a very strong general heuristic. The Iterative Deepening paired with Dynamic Move Re-Ordering that starts out as the central move ordering could be improved on by the move ordering being calculated at each layer of the tree, as opposed to once at the beginning of each iterative deepening call to Minimax. With this improvement its efficiency would likely surpass the static center Move Re-Ordering + Alpha-Beta pruning. The Selective Deepening optimization did in fact make the Connect 4 AI a

better player. It could be improved on by allowing the search to continue extending the search of a threat path until it was no longer a threat, as opposed to the single layer extension that was implemented in this project.

USE OF AI

Code Generation

The Claude Sonnet 4.6 Model was used for the initial implementation of the projects argument parsing, Board class, and the visual library set up of the realtime Connect 4 board. This was done by providing the model with a detailed root level .md file and a few specific .md task files. Other than that the majority of the code was written by hand to better understand the Minimax algorithm and its optimization techniques.

Writeup

The Claude Sonnet 4.6 Model was used for all the Latex structural code and library inclusions needed for this document. The content of each section of the document was written by hand and are my own words. The model was used to help with the formatting of the tables and figures, but the data content of the tables were gathered manually by running the program and recording the data. The model was also used for grammar and spelling checks of the section content.